



COMP9444 Neural Networks and Deep Learning

Assignment 1

Term 3, 2024

Submitted by

zID: z5319476

Name: Jiayang Jiang

I declare that:

This assessment item is entirely my own original work, except where I have acknowledged use of source material [such as books, journal articles, other published material, the Internet, and the work of other student/s or any other person/s.

This assessment item has not been submitted for assessment for academic credit in this, or any other course, at UNSW or elsewhere.

I understand that:

The assessor of this assessment item may, for the purpose of assessing this item, reproduce this assessment item and provide a copy to another member of the University.

The assessor may communicate a copy of this assessment item to a plagiarism checking service (which may then retain a copy of the assessment item on its database for the purpose of future plagiarism checking).

I certify that I have read and understood the University Rules in respect of Student Academic Misconduct.

Jiayang Jiang, z5319476, and date

Part 1: Japanese Character Recognition

1. Answer question 1

The confusion matrix is:

```
[[763.  5.  8. 12. 30. 66.  2. 63. 32. 19.]
 [ 7. 670. 107. 18. 30. 21. 58. 12. 25. 52.]
 [ 7.  62. 687. 26. 28. 21. 48. 37. 45. 39.]
 [ 4.  37.  59. 757. 15. 57. 15. 18. 26. 12.]
 [59.  54.  77. 19. 627. 20. 33. 34. 22. 55.]
 [ 8.  28. 124. 16. 19. 727. 27.  9. 31. 11.]
 [ 5.  23. 146. 10. 26.  25. 723. 20.  9. 13.]
 [16.  30.  27. 13.  82. 16.  55. 624. 89. 48.]
 [10.  38.  92. 42.  9.  31.  45.  7. 704. 22.]
 [ 8.  52.  85.  3.  52.  30. 19.  32.  39. 680.]]
```

Test set: Average loss: 1.0101, Accuracy: 6962/10000 (70%)

2. Answer question 2

Different number of hidden nodes and accuracy:

The number of hidden nodes	Accuracy
40	7985/10000 (80%)
80	8272/10000 (83%)
160	8457/10000 (85%)
240	8492/10000 (85%)
220	8443/10000 (84%)
230	8441/10000 (84%)

I chose the value 240 since it achieved highest accuracy (85%).

The confusion matrix is:

```
[[854.  2.  3.  6. 28. 29.  3. 37. 32.  6.]
 [ 5. 804. 36.  3. 17. 13. 63.  4. 26. 29.]
 [ 8. 17. 842. 34. 11. 15. 24. 11. 18. 20.]
 [ 2.  8.  22. 923.  2. 15.  7.  2.  8. 11.]
 [42. 22.  22.  3. 820.  7. 28. 17. 22. 17.]
 [ 8. 10.  83. 11. 11. 834. 21.  2. 15.  5.]
 [ 3. 11.  55.  6. 15.  5. 891.  6.  2.  6.]
 [15. 15.  18.  5. 24. 10. 33. 829. 21. 30.]
 [10. 27.  32. 41.  5.  8. 28.  5. 839.  5.]
 [ 3. 11.  45.  5. 27.  5. 21. 13. 14. 856.]]
```

Test set: Average loss: 0.4957, Accuracy: 8492/10000 (85%)

Calculation of the total number of independent parameters:

First layer: `self.hidden_layer = nn.Linear(28 * 28, 240)`

The input size is $28 \times 28 = 784$, and the output size is 240. Therefore,

- Number of weight parameters = $240 * 784 = 188160$
- Number of bias parameters = 240
- Number of independent parameters for the first layer = $188160 + 240 = 188400$

Second layer: `self.output_layer = nn.Linear(240, 10)`

The input size is 240, and the output size is 10. Therefore,

- Number of weight parameters = $10 * 240 = 2400$
- Number of bias parameters = 10
- Number of independent parameters for the second layer = $2400 + 10 = 2410$

As a result, total number of independent parameters in the network is

$$188400 + 2410 = 190810.$$

3. Answer question 3

I set seed to have more reproducible results. (`torch.manual_seed(42)`)

I used Dropout to reduce overfitting and max pooling

The confusion matrix is:

```
[[965.  3.  3.  0. 18.  1.  0.  7.  1.  2.]
 [ 1. 920.  6.  1. 11.  1. 35.  5. 10. 10.]
 [10.  9. 885. 39.  9. 11. 20.  8.  5.  4.]
 [ 1.  3. 11. 975.  0.  6.  2.  1.  0.  1.]
 [27.  3.  4.  7. 927.  2. 10.  7. 11.  2.]
 [ 3.  5. 33.  7.  4. 931.  9.  2.  3.  3.]
 [ 4.  2. 22.  2.  6.  2. 959.  2.  1.  0.]
 [ 8.  2.  5.  1. 11.  1.  2. 948.  7. 15.]
 [ 5.  4.  6.  9.  7.  1.  1.  0. 967.  0.]
 [ 9.  4. 13.  2.  9.  1.  2.  3.  1. 956.]]
```

Test set: Average loss: 0.2038, Accuracy: 9433/10000 (94%)

Calculation of the total number of independent parameters:

First Convolutional Layer:

- Input channels: 1
- Output channels: 32
- Kernel size: 5×5
- Number of weight parameters = $1 \times 32 \times 5 \times 5 = 800$
- Number of bias parameters = 32
- Total number of parameters = $800 + 32 = 832$

Second Convolutional Layer:

- Input channels: 32
- Output channels: 64
- Kernel size: 5×5
- Number of weight parameters = $32 \times 64 \times 5 \times 5 = 51200$
- Number of bias parameters = 64
- Total number of parameters = $51200 + 64 = \mathbf{51264}$

Fully Connected Layer:

- Number of input nodes: $64 \times 7 \times 7 = 3136$
- Number of output nodes: 128
- Number of weight parameters = $3136 \times 128 = 401408$
- Number of bias parameters = 128
- Total number of parameters = $401408 + 128 = \mathbf{401536}$

Output Layer:

- Number of input nodes: 128
- Number of output nodes: 10
- Number of weight parameters = $128 \times 10 = 1280$
- Number of bias parameters = 10
- Total number of parameters = $1280 + 10 = \mathbf{1290}$

As a result, total number of independent parameters in the network is

$$832 + 51264 + 401536 + 1290 = 454922$$

4. Answer question 4

(a) the relative accuracy of the three models

NetLin (Linear Model):

- This model performs a simple linear transformation on the image pixels and then applies log softmax. The final test accuracy is around **70%**. Since this model does not have the capacity to learn complex features, it performs relatively poorly, especially for distinguishing characters with subtle differences.

NetFull (Two-Layer Fully Connected Model):

- This model introduces one hidden layer with the tanh activation function, allowing it to capture more complex patterns compared to the linear model. The accuracy increases to 85%, indicating a significant improvement due to the non-linear capacity of the model.

NetConv (Convolutional Neural Network):

- The CNN model, with its two convolutional layers, is highly suitable for image recognition tasks. It extracts spatial features from the images through filters, pooling, and activation functions. The final accuracy is **94%**, the highest among the three models. CNN's ability to capture spatial hierarchies and local patterns in the image makes it particularly effective.

Therefore, for accuracy, **NetConv > NetFull > NetLin**.

(b) the number of independent parameters in each of the three models

NetLin (Linear Model):

- This model has only one fully connected layer. The total number of independent parameters is $28 \times 28 \times 10 + 10$ (bias) = $7840 + 10 = \mathbf{7850}$.

NetFull (Two-Layer Fully Connected Model):

- With one hidden layer and one output layer, the total number of independent parameters is **190810**. The addition of the hidden layer introduces more parameters, but the overall complexity is still relatively low compared to the convolutional network.

NetConv:

- The CNN model has two convolutional layers followed by a fully connected layer, resulting in a total of **454922** independent parameters. The larger number of parameters comes from the convolutional layers, which have many filters applied to multiple input channels, allowing the network to capture more detailed patterns

(c) the confusion matrix for each model: which characters are most likely to be mistaken for which other characters, and why?

The definition: (0="o", 1="ki", 2="su", 3="tsu", 4="na", 5="ha", 6="ma", 7="ya", 8="re", 9="wo")

NetLin (Linear Model):

```
[[763.  5.  8. 12. 30. 66.  2. 63. 32. 19.]
 [ 7. 670. 107. 18. 30. 21. 58. 12. 25. 52.]
 [ 7.  62. 687. 26. 28. 21. 48. 37. 45. 39.]
 [ 4.  37.  59. 757. 15. 57. 15. 18. 26. 12.]
 [59.  54.  77. 19. 627. 20. 33. 34. 22. 55.]
 [ 8.  28. 124. 16. 19. 727. 27.  9. 31. 11.]
 [ 5.  23. 146. 10. 26. 25. 723. 20.  9. 13.]
 [16.  30.  27. 13. 82. 16. 55. 624. 89. 48.]
 [10.  38.  92. 42.  9. 31. 45.  7. 704. 22.]
 [ 8.  52.  85.  3. 52. 30. 19. 32. 39. 680.]]
```

- In the confusion matrix, characters such as **1 ("ki")** and **2 ("su")** are often confused with each other, as evidenced by 107 instances of misclassification. This likely occurs because these characters share similar shapes, and the linear model (NetLin), which only relies on pixel intensity combinations, lacks the capacity to differentiate these subtle visual differences. Similarly, **5 ("ha")** is frequently misclassified as **2 ("su")** (124 instances), and **6 ("ma")** is confused with **2 ("su")** 146 times. These confusions highlight the limitations of the linear model in distinguishing between characters that have overlapping or structurally similar features, as it cannot effectively capture higher-level patterns or shapes in the data.

NetFull (Two-Layer Fully Connected Model):

```
[[854.  2.  3.  6. 28. 29.  3. 37. 32.  6.]
 [ 5.804. 36.  3. 17. 13. 63.  4. 26. 29.]
 [ 8. 17.842. 34. 11. 15. 24. 11. 18. 20.]
 [ 2.  8. 22.923.  2. 15.  7.  2.  8. 11.]
 [42. 22. 22.  3.820.  7. 28. 17. 22. 17.]
 [ 8. 10. 83. 11. 11.834. 21.  2. 15.  5.]
 [ 3. 11. 55.  6. 15.  5.891.  6.  2.  6.]
 [15. 15. 18.  5. 24. 10. 33.829. 21. 30.]
 [10. 27. 32. 41.  5.  8. 28.  5.839.  5.]
 [ 3. 11. 45.  5. 27.  5. 21. 13. 14.856.]]
```

- In the confusion matrix for the **NetFull (Two-Layer Fully Connected Model)**, characters like **1 ("ki")** and **2 ("su")** still show confusion, with **36** instances of misclassification. While this is an improvement from NetLin, the model still struggles with subtle shape differences. Similarly, **5 ("ha")** is misclassified as **2 ("su")** **83** times, and **6 ("ma")** as **2 ("su")** **55** times, though these rates have decreased. The added hidden layer improves performance by introducing non-linearity, but the model still finds it challenging to distinguish between structurally similar characters, indicating that fully connected layers alone are insufficient for capturing detailed features in these cases.

NetConv (Convolutional Neural Network):

```
[[965.  3.  3.  0. 18.  1.  0.  7.  1.  2.]
 [ 1.920.  6.  1. 11.  1. 35.  5. 10. 10.]
 [10.  9.885. 39.  9. 11. 20.  8.  5.  4.]
 [ 1.  3. 11.975.  0.  6.  2.  1.  0.  1.]
 [27.  3.  4.  7.927.  2. 10.  7. 11.  2.]
 [ 3.  5. 33.  7.  4.931.  9.  2.  3.  3.]
 [ 4.  2. 22.  2.  6.  2.959.  2.  1.  0.]
 [ 8.  2.  5.  1. 11.  1.  2.948.  7. 15.]
 [ 5.  4.  6.  9.  7.  1.  1.  0.967.  0.]
 [ 9.  4. 13.  2.  9.  1.  2.  3.  1.956.]]
```

- In the confusion matrix for **NetConv (Convolutional Neural Network)**, the misclassifications have significantly decreased compared to the previous models. **1 ("ki")** is confused with **2 ("su")** only **6** times, and **5 ("ha")** is misclassified as **2 ("su")** **33** times, which is a marked improvement from the two-layer fully connected model. Similarly, **6 ("ma")** is confused with **2 ("su")** only **22** times. The convolutional layers help the model capture spatial features more effectively, reducing confusion between characters with similar shapes. One of the most frequent misclassifications is **2 ("su")** being confused with **3 ("tsu")**, with **39** instances. This likely occurs because these two characters have structural similarities that the model still finds challenging to fully differentiate, even with convolutional layers. While the CNN performs significantly better overall, this particular confusion highlights the ongoing difficulty in distinguishing visually similar characters. Overall, the CNN demonstrates a much higher accuracy by leveraging spatial feature extraction, which significantly reduces misclassification rates.

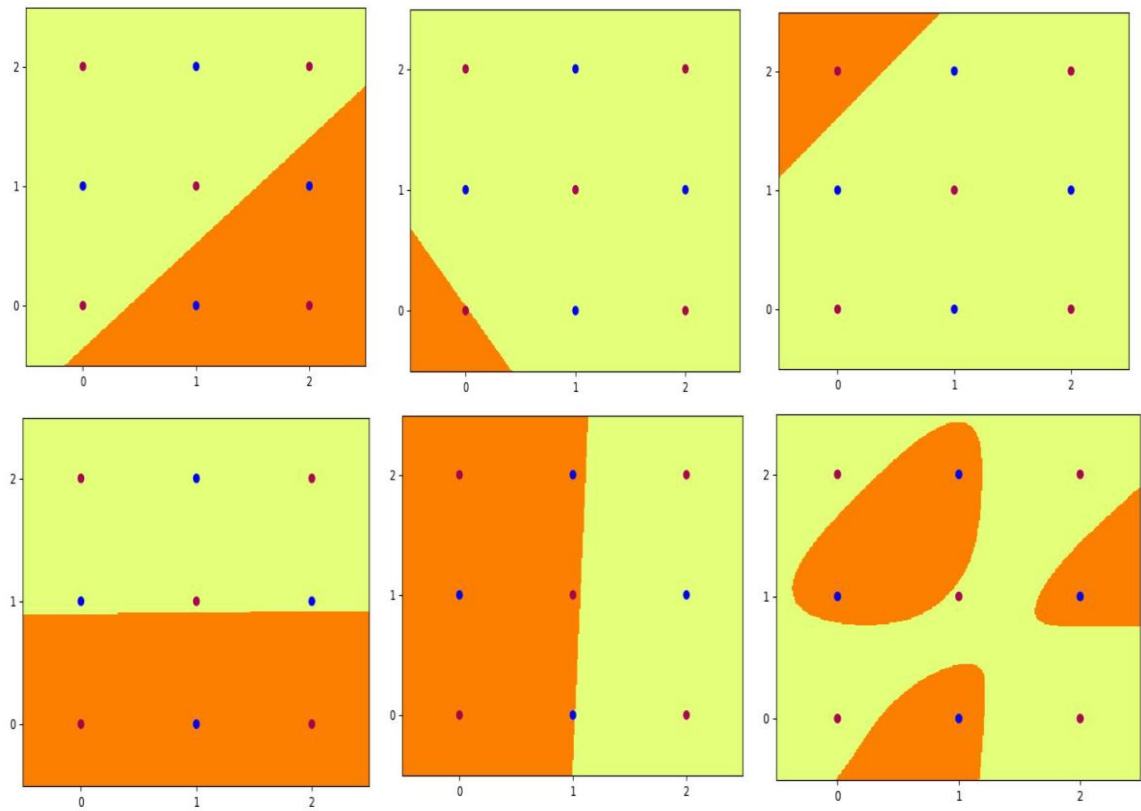
Summary:

- **Relative Accuracy:** NetLin < NetFull < NetConv. The more complex the model, the better it can extract meaningful features from the images, leading to higher accuracy.
- **Number of Independent Parameters:** NetConv has the most parameters due to its convolutional layers, which helps in capturing more information from the input but increases computational complexity.
- **Confusion Matrix Analysis:** The confusion between characters in the simpler models is largely due to their visual similarity, and as the models become more complex, the misclassification rates decrease. CNN is much better at learning and distinguishing between characters, but certain characters with similar structures, like **2 ("su")** and **3 ("tsu")**, can still be confused.

Part 2: Multi-Layer Perceptron

1. Answer question 1

hid_5_?.jpg and out_5.jpg:



2. Answer question 2

Hand-crafted weights and biases:

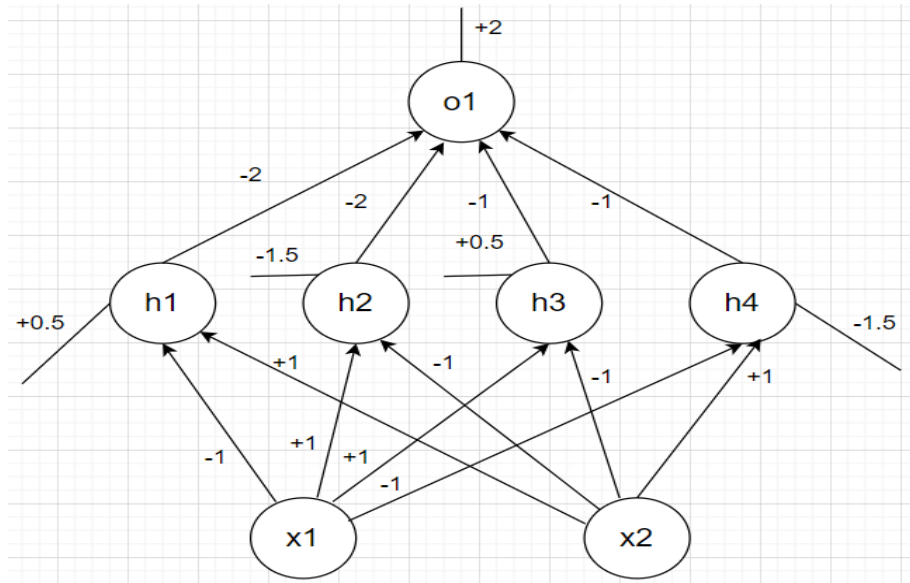
$\text{in_hid_weight} = [[-1, 1], [1, -1], [1, -1], [-1, 1]]$

$\text{hid_bias} = [0.5, -1.5, 0.5, -1.5]$

$\text{hid_out_weight} = [[-2, -2, -1, -1]]$

$\text{out_bias} = [2]$

Diagram of the network:



Equations for the dividing line:

For Hidden Node 1:

$$-x_1 + x_2 + 0.5 = 0 \text{ (simplifies to: } x_2 = x_1 - 0.5 \text{)}$$

For Hidden Node 2:

$$x_1 - x_2 - 1.5 = 0 \text{ (simplifies to: } x_2 = x_1 - 1.5 \text{)}$$

For Hidden Node 3:

$$x_1 - x_2 + 0.5 = 0 \text{ (simplifies to: } x_2 = x_1 + 0.5 \text{)}$$

For Hidden Node 4:

$$-x_1 + x_2 - 1.5 = 0 \text{ (simplifies to: } x_2 = x_1 + 1.5 \text{)}$$

Activation Table:

Output Node = $H_1 \cdot \text{weight}_1 + H_2 \cdot \text{weight}_2 + H_3 \cdot \text{weight}_3 + H_4 \cdot \text{weight}_4 + \text{bias}$

Training items	x1	x2	Hidden Node 1	Hidden Node 2	Hidden Node 3	Hidden Node 4	Output Node
1	0	0	1	0	1	0	0
2	0	1	1	0	0	0	1
3	0	2	1	0	0	1	0
4	1	0	0	0	1	0	1
5	1	1	1	0	1	0	0
6	1	2	1	0	0	0	1
7	2	0	0	1	1	0	0
8	2	1	0	0	1	0	1
9	2	2	1	0	1	0	0

3. Answer question 3

Rescale weights and biases by 10:

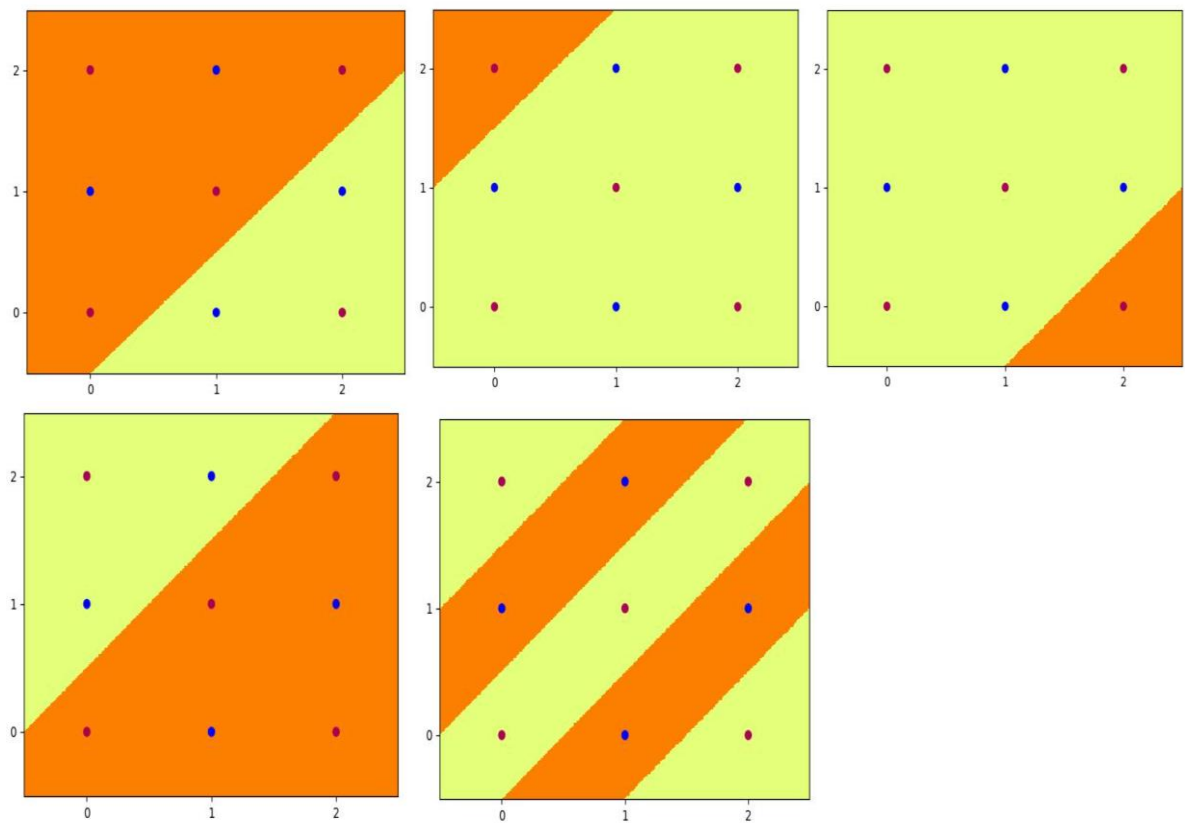
`in_hid_weight = [[-10, 10], [10, -10], [10, -10], [-10, 10]]`

`hid_bias = [5, -15, 5, -15]`

`hid_out_weight = [[-20, -20, -10, -10]]`

`out_bias = [20]`

`hid_4_?.jpg` and `out_4.jpg`:



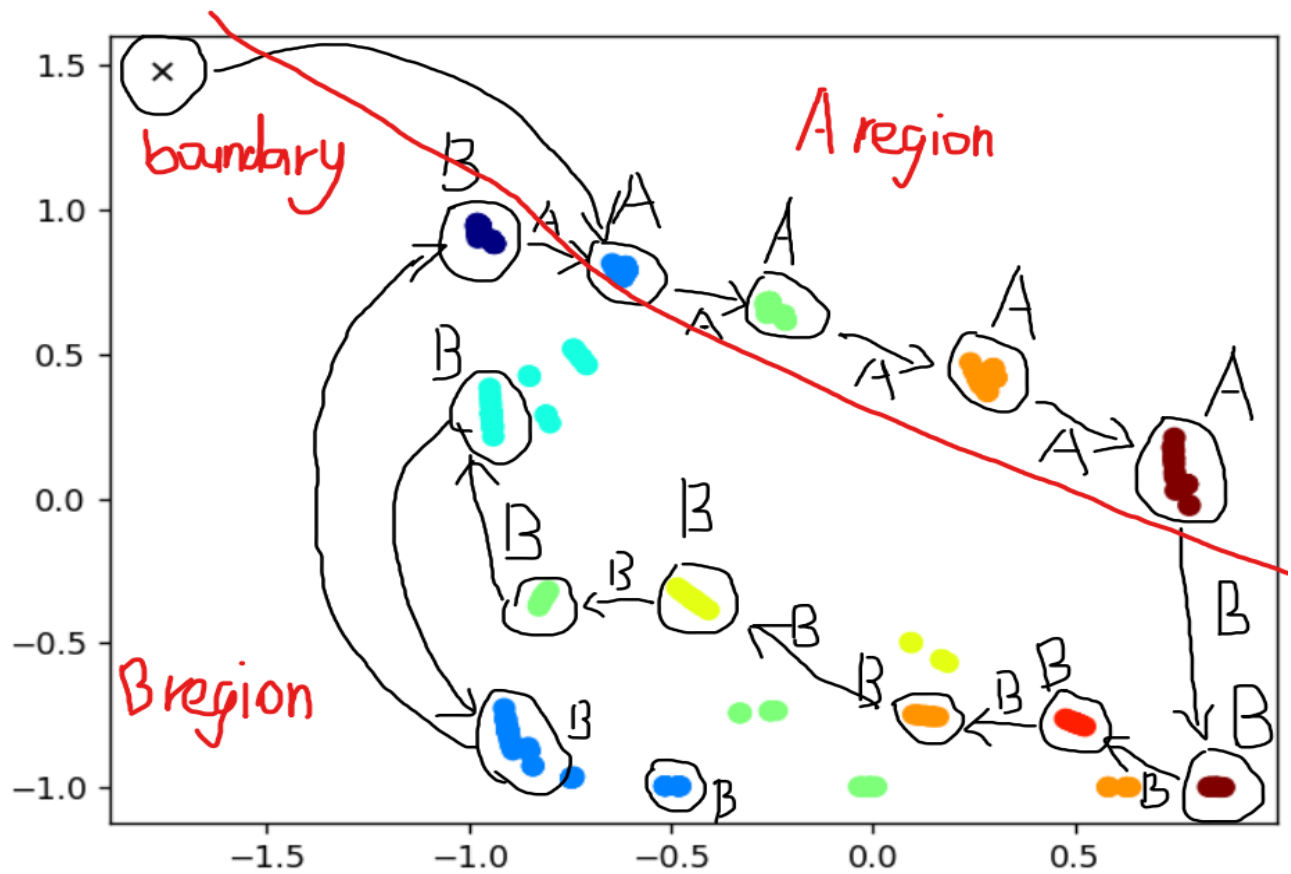
Part 3: Hidden Unit Dynamics for Recurrent Networks

1. Answer question 1

The hidden activations of my sequence:

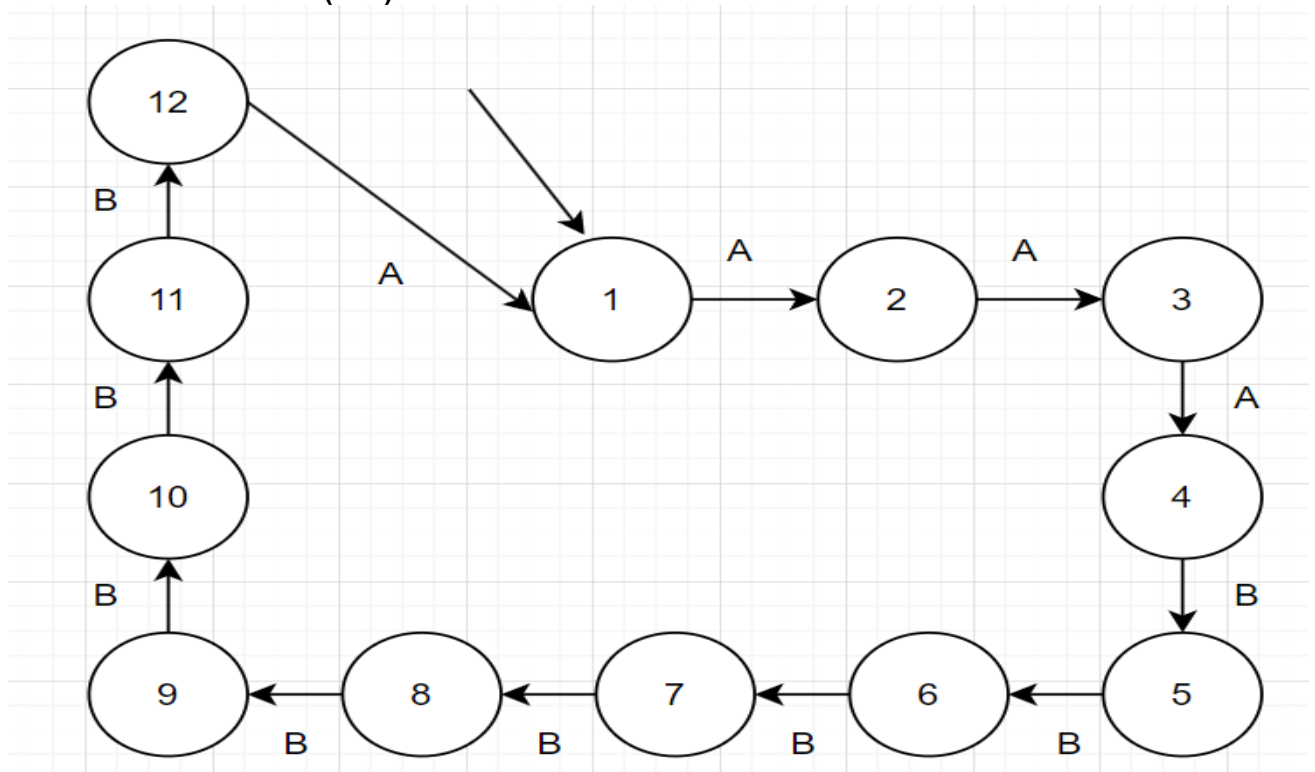
```
symbol= ABBAAABBBBBBAAAABBBBBBBBBBAAAABBBBBBBBABBA
hidden activations and output probabilities:
A [-0.61  0.79] [0.8 0.2]
B [-0.48 -1.  ] [0. 1.]
B [-0.94  0.88] [0.85 0.15]
A [-0.61  0.81] [0.82 0.18]
A [-0.22  0.62] [0.64 0.36]
A [0.3  0.45] [0.48 0.52]
B [ 0.64 -1.  ] [0. 1.]
B [ 0.18 -0.57] [0. 1.]
B [-0.24 -0.73] [0. 1.]
B [-0.8  0.26] [0.07 0.93]
B [-0.85 -0.86] [0. 1.]
B [-0.98  0.94] [0.89 0.11]
A [-0.63  0.78] [0.79 0.21]
A [-0.26  0.67] [0.72 0.28]
A [0.27 0.4 ] [0.38 0.62]
A [0.74 0.17] [0.16 0.84]
B [ 0.86 -1.  ] [0. 1.]
B [ 0.51 -0.78] [0. 1.]
B [ 0.15 -0.75] [0. 1.]
B [-0.43 -0.37] [0. 1.]
B [-0.81 -0.33] [0. 1.]
B [-0.94  0.25] [0.06 0.94]
B [-0.91 -0.77] [0. 1.]
B [-0.98  0.92] [0.87 0.13]
```

Annotated image base on given hidden activations:



2. Answer question 2

Finite state machine($n=4$):



3. Answer question 3

When the RNN processes the $anb2n$ task, it determines n by counting the number of A's. Once the first B is encountered, the hidden unit activations change, signaling that the model knows how many A's came before and can now predict exactly $2n$ B's. After predicting all the B's, the model resets to its initial state and correctly predicts the next A of the new sequence. During the process, the hidden units shift from a non-deterministic state for A to a deterministic state for B, ensuring accurate predictions.

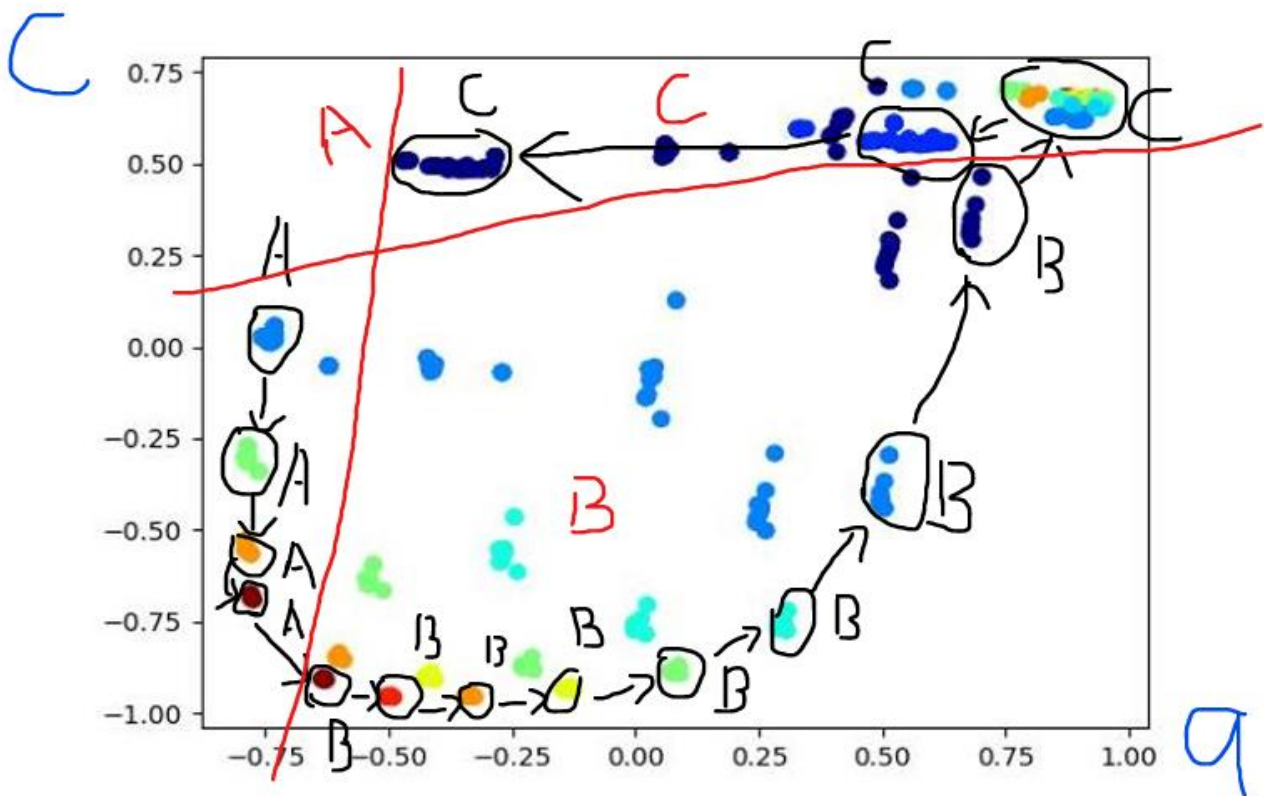
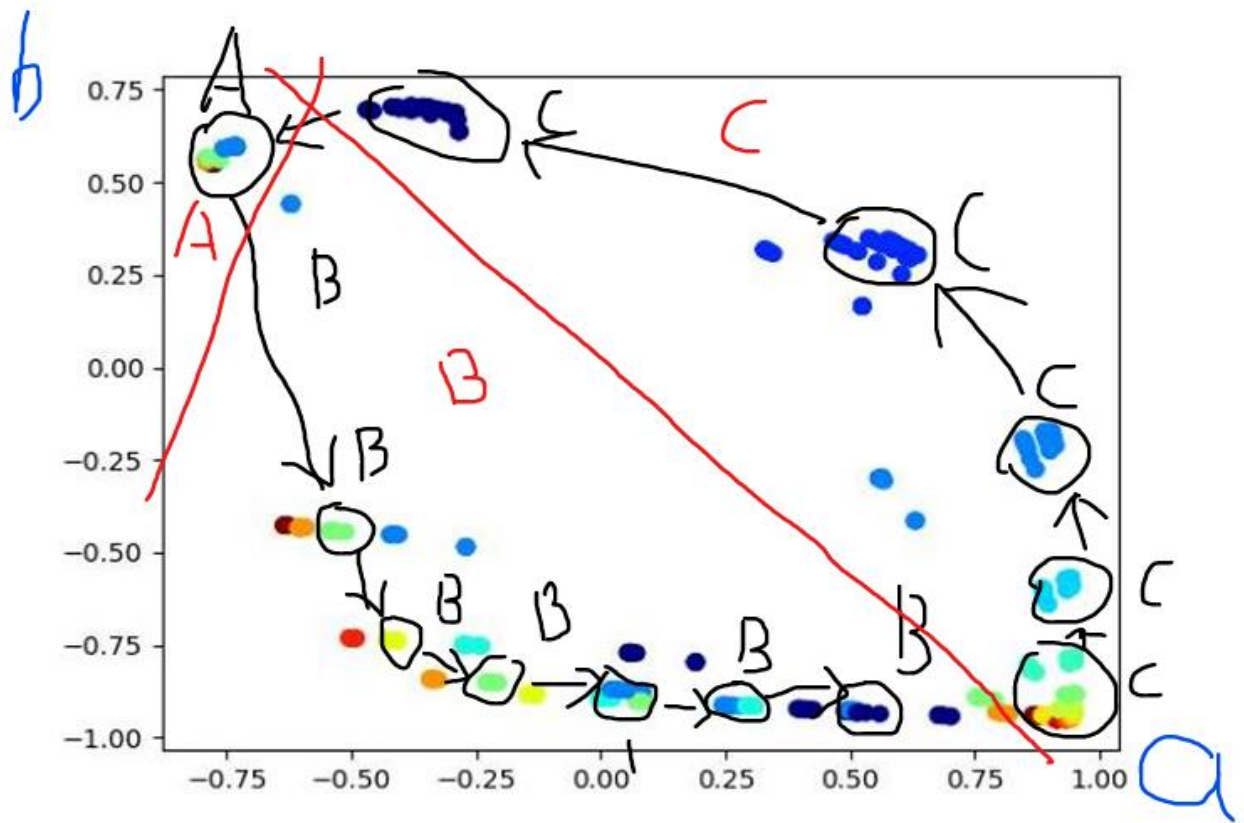
4. Answer question 4

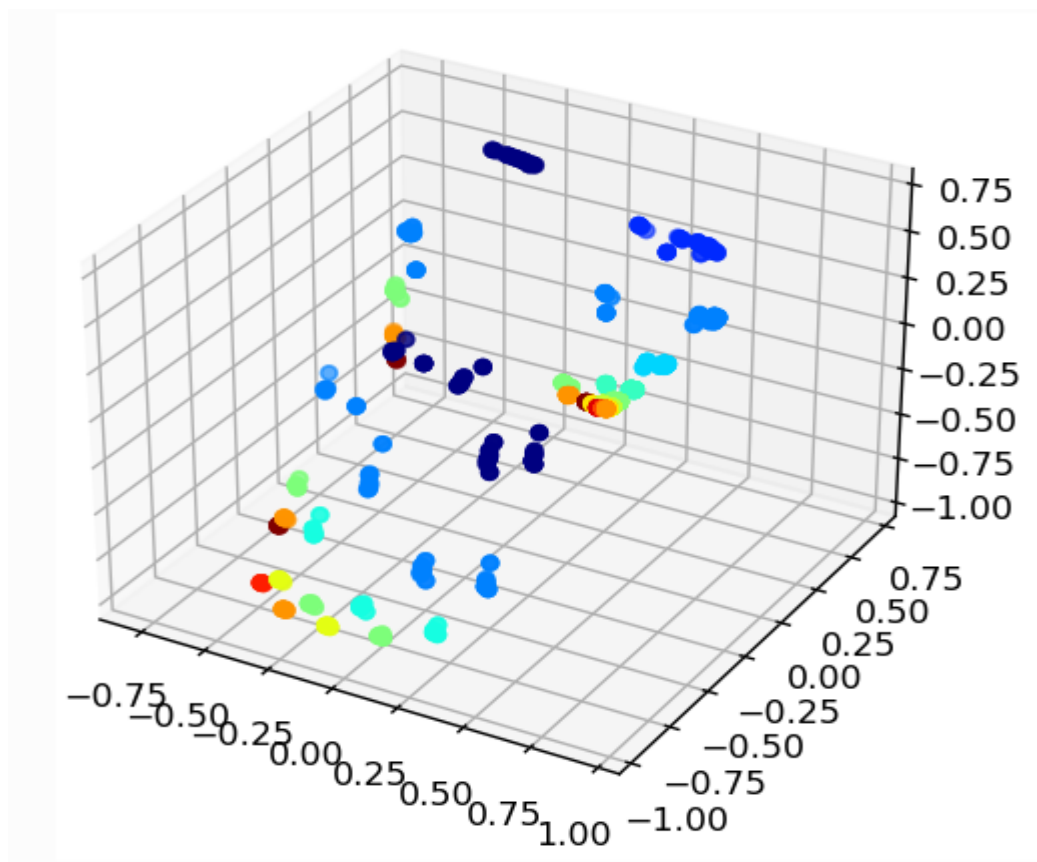
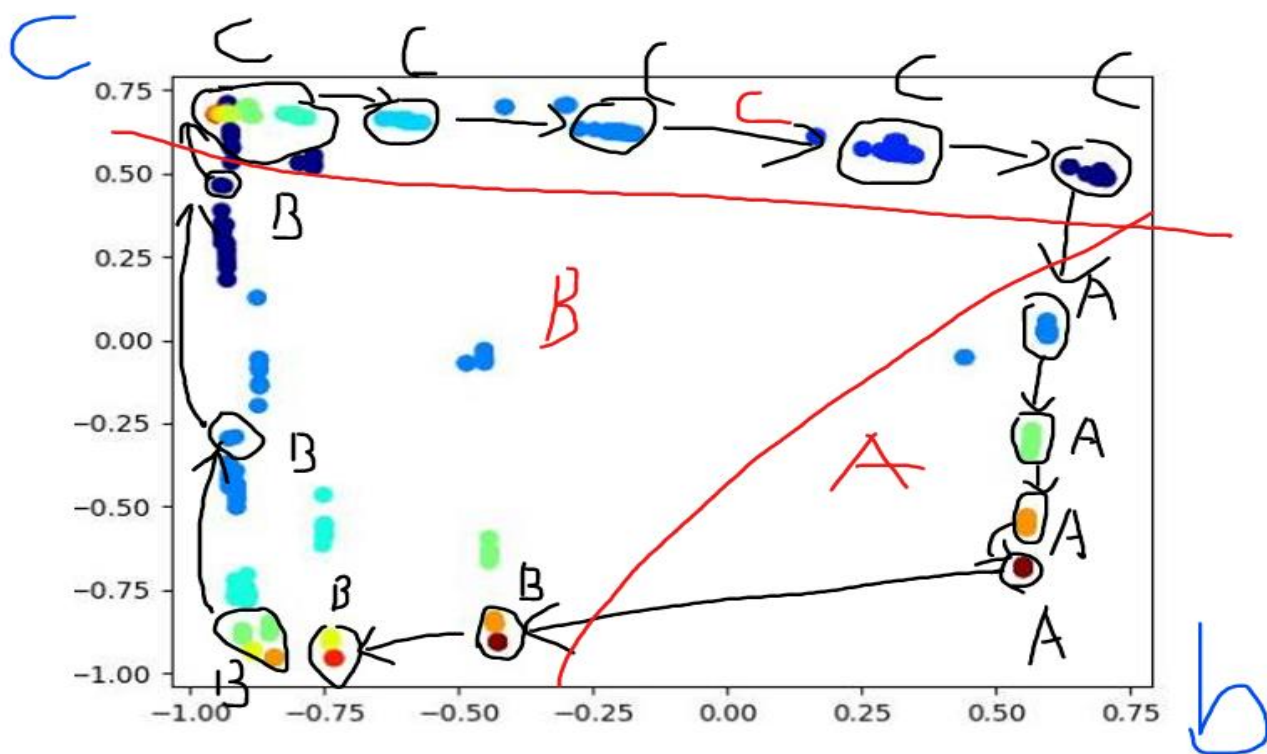
I chose epoch number 50, this language model has 3 axes.

The hidden activations of my sequence:

```
symbol= ABBCCCAAAABBBBCCCCCCCCCAABBBBCCCCCCCCCABBCCCAABBBCC
hidden activations and output probabilities:
A [-0.62  0.44 -0.05] [0.66 0.33 0.01]
B [-0.27 -0.49 -0.07] [0.03 0.92 0.05]
B [ 0.19 -0.8  0.53] [0.01 0.06 0.93]
C [ 0.63 -0.41  0.7 ] [0. 0. 1.]
C [0.52 0.17 0.61] [0.01 0. 0.98]
C [-0.28 0.64 0.52] [0.85 0.02 0.13]
A [-0.73 0.59 0.06] [0.85 0.14 0. ]
A [-0.79 0.57 -0.27] [0.61 0.39 0. ]
A [-0.79 0.56 -0.53] [0.36 0.64 0. ]
A [-0.78 0.55 -0.67] [0.24 0.76 0. ]
B [-0.63 -0.43 -0.9 ] [0. 1. 0.]
B [-0.5  -0.73 -0.95] [0. 1. 0.]
B [-0.33 -0.84 -0.95] [0. 1. 0.]
B [-0.13 -0.88 -0.93] [0. 1. 0.]
B [ 0.09 -0.9 -0.87] [0. 1. 0.]
B [ 0.31 -0.92 -0.72] [0. 0.99 0.01]
B [ 0.51 -0.93 -0.3 ] [0. 0.69 0.31]
B [ 0.7  -0.94 0.46] [0. 0.01 0.99]
C [ 0.88 -0.94 0.68] [0. 0. 1.]
C [ 0.92 -0.95 0.68] [0. 0. 1.]
C [ 0.93 -0.95 0.68] [0. 0. 1.]
C [ 0.94 -0.95 0.68] [0. 0. 1.]
C [ 0.95 -0.94 0.68] [0. 0. 1.]
C [ 0.95 -0.93 0.68] [0. 0. 1.]
C [ 0.95 -0.88 0.67] [0. 0. 1.]
C [ 0.95 -0.79 0.67] [0. 0. 1.]
C [ 0.94 -0.59 0.65] [0. 0. 1.]
C [ 0.91 -0.2  0.62] [0. 0. 1.]
C [0.62 0.32 0.56] [0.02 0. 0.98]
C [-0.31 0.69 0.48] [0.89 0.02 0.09]
A [-0.73 0.6 0.02] [0.84 0.16 0. ]
```

Annotated image (hidden activations):



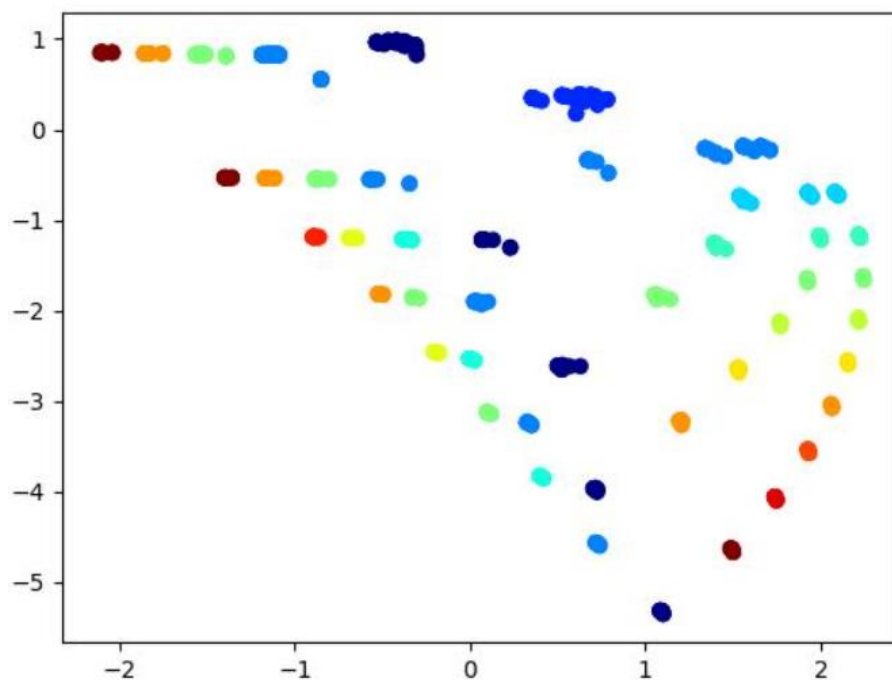


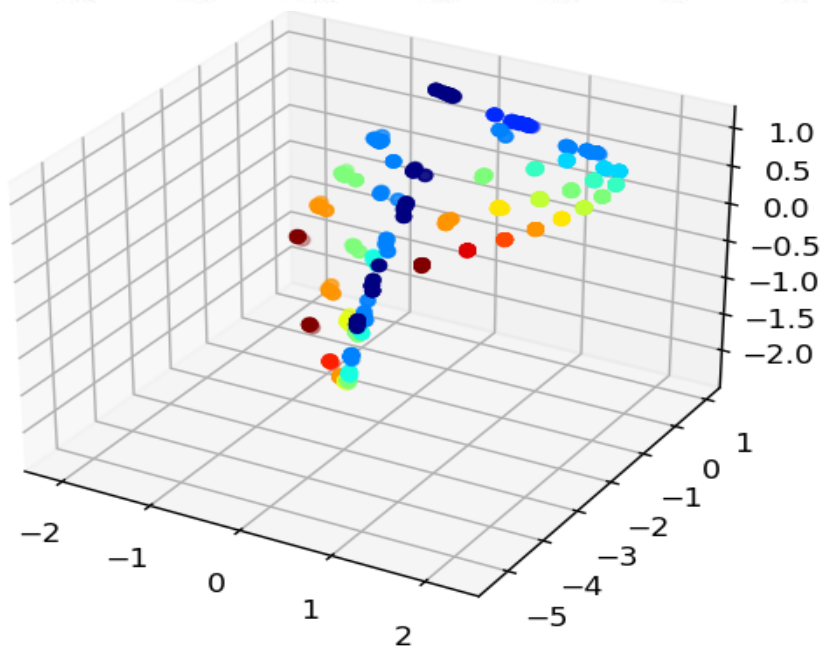
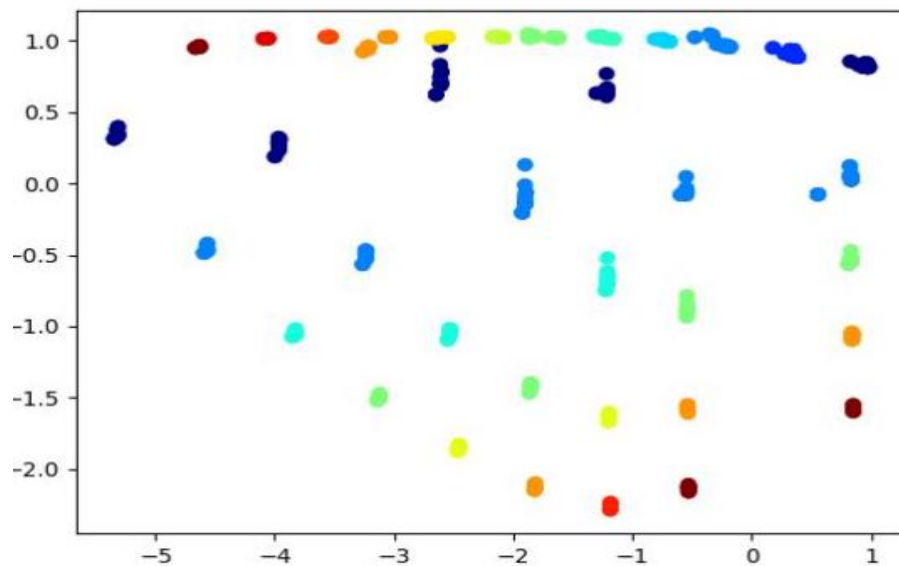
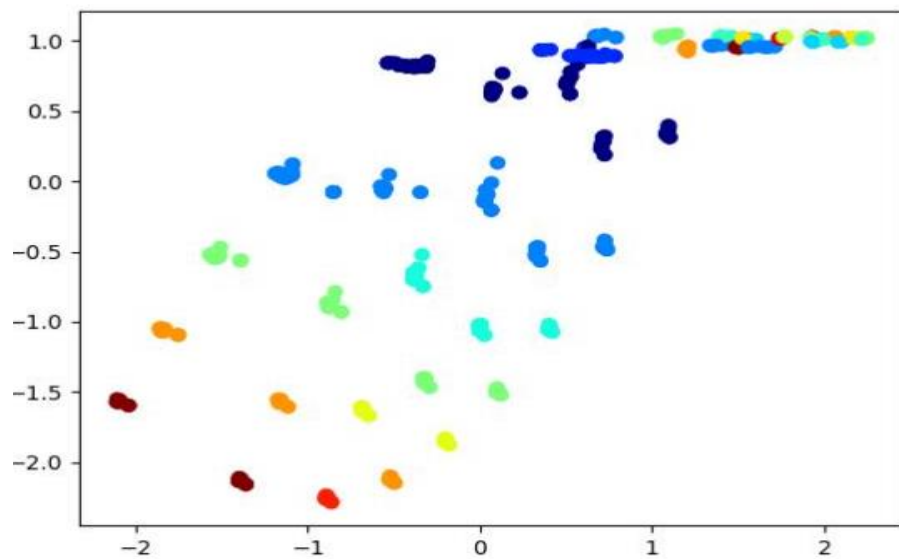
5. Answer question 5

To print out the context units, I modified the seq_plot.py and seq_model.py file.

```
Epoch 9: Context Units (c_t):  
[[-0.85  0.55 -0.08]  
 [-1.39  0.81 -0.56]  
 [-1.75  0.84 -1.09]  
 [-2.04  0.85 -1.6 ]  
 [-1.36 -0.53 -2.16]  
 [-0.86 -1.19 -2.28]  
 [-0.5  -1.82 -2.15]  
 [-0.18 -2.47 -1.87]  
 [ 0.12 -3.14 -1.52]  
 [ 0.42 -3.85 -1.07]  
 [ 0.74 -4.59 -0.49]  
 [ 1.1  -5.35  0.31]  
 [ 1.5  -4.67  0.95]  
 [ 1.75 -4.09  1.02]  
 [ 1.93 -3.57  1.03]  
 [ 2.07 -3.07  1.03]  
 [ 2.16 -2.59  1.03]  
 [ 2.22 -2.12  1.02]  
 [ 2.25 -1.66  1.02]  
 [ 2.23 -1.2   1.01]
```

Context units plots:





In LSTM network, the context units act as a memory cell that carries important information across time steps. These units help the model retain long-term dependencies, which is crucial for counting the number of A's, B's, and C's and predicting the sequences in the ab^2nc^3n language.

The context units evolve smoothly over time and store critical information, such as how many A's, B's, and C's have been seen. This enables the LSTM to track progress through the sequence.

LSTM network has three main gates: input, forget, and output gates. They control the flow of information to and from the context units, ensuring the LSTM focuses on relevant inputs and discards irrelevant information when necessary.

Working process:

- The context units, or memory cells, store the internal state of the network. This state is updated at each time step through interactions with the input gate, forget gate, and output gate.
- The **input gate** decides what new information to store in the memory cell.
- The **forget gate** controls how much of the existing memory should be kept or discarded.
- The **output gate** determines how much of the memory cell content should be passed to the hidden state and used to make predictions at that particular time step.

Dynamics of the Hidden Units:

The **hidden units** provide the immediate output of the LSTM at each time step. Their activations depend on both the current input and the internal context, allowing them to handle the prediction of each symbol in the sequence. Also, through annotated image and the context units images, we can analyze the prediction task.

- **Processing A's:** As the LSTM processes A's, the context units update to store the count of A's seen so far. The hidden unit activations shift to reflect this, predicting the next character based on the updated memory.
- **Handling B's and C's:** Once the first B is encountered, the context units store the count of A's, enabling the LSTM to predict exactly $2n$ B's. After predicting the B's, the context units transition to handle the C's, ensuring that the correct $3n$ C's are predicted. The hidden units track these transitions, reflecting the shift from handling A's to B's, and finally to C's, ensuring the correct sequence is followed.